Ocean modeling at the dawn of AI

Risks and opportunities for usage, development, and parameterization

Gregory Wagner

NEMO 2026 Developers' Workshop · 13 October 2026

Gregory Wagner

Head of Numerical Weather Prediction, Aeolus Labs

I develop Oceananigans, Breeze, and NumericalEarth.
At Aeolus Labs, we're building a differentiable,
hardware-portable numerical weather
prediction system based on Oceananigans.

Breeze is our atmosphere model.

NumericalEarth is our Earth system modeling framework.

glwagner.github.io

AI, ML, LLMs—and the tools around them

FROM BROAD FIELD TO SPECIFIC MODEL

AI · artificial intelligence

The broad field: learning, reasoning, planning.

ML · machine learning

Models fitted from data: trees, kernels, networks…

Deep learning

Multilayer neural networks, typically trained with gradient-based optimization.

LLMs · large language models

Usually transformers, pretrained at scale on vast collections of text and code.

WHAT WE ACTUALLY USE

An LLM inside a software system

LLM
the model

\+

Harness

Context and instructions
Tools: files, shell, search
Permissions and state

Read → act → inspect → repeat

An agent lets the model choose which steps and tools to use.

Here, "AI" often means an LLM-powered assistant.
ML also includes learned parameterizations and emulators.

IBM: terminology · LeCun et al. (2015) · Vaswani et al. (2017) · Brown et al. (2020) · Anthropic: Building effective agents (2024)

Large language models are extraordinary technology.

Opportunities

Faster implementation

Turn ideas into working experiments.

Wider participation

Help newcomers navigate complex code.

Risks

Loss of attribution

Human contributions become obscured.

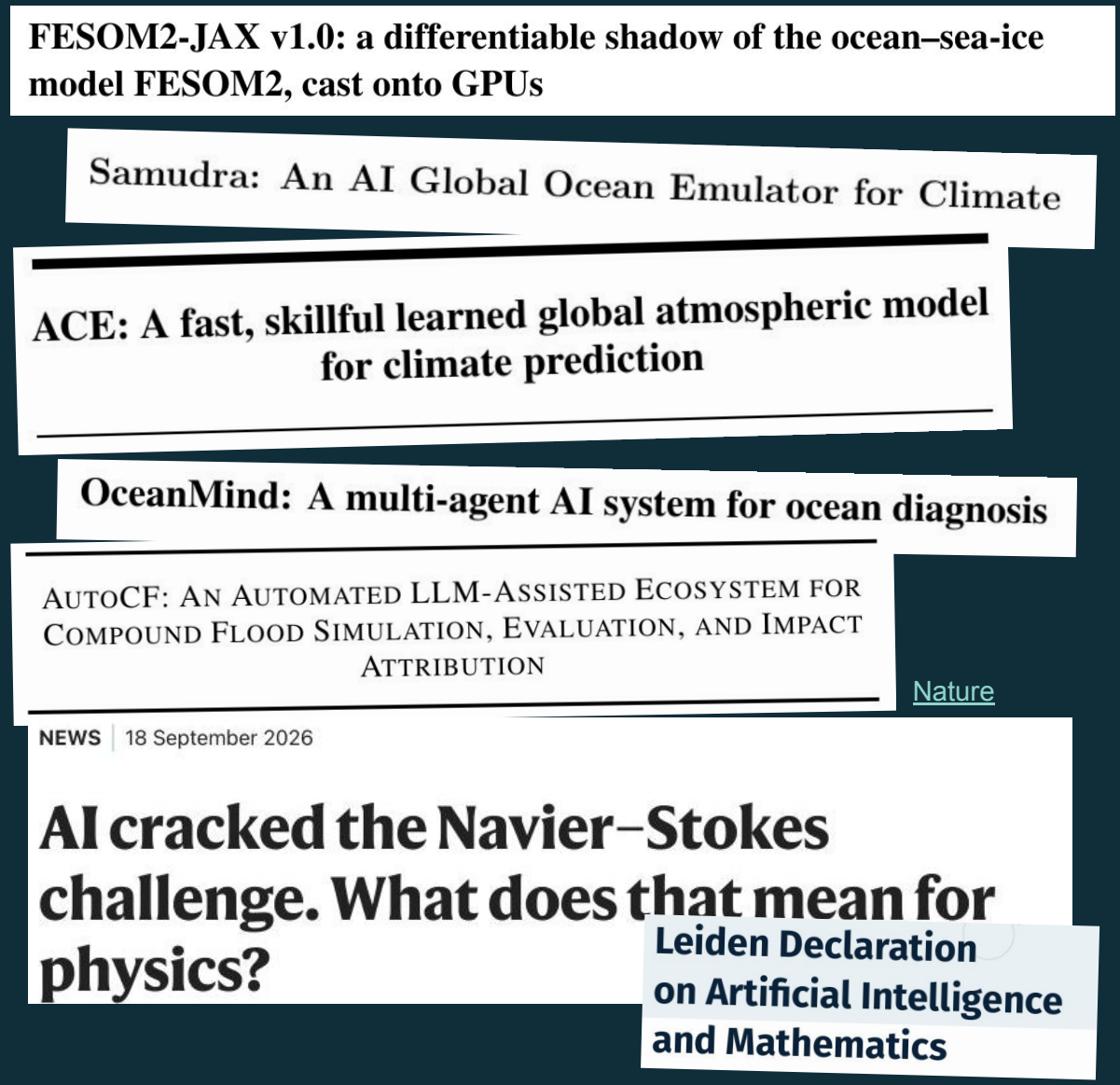
Loss of human understanding and control

Tools and vendors shape research priorities.

Porting ≠ progress

Are we rewriting or building new code?

Leiden Declaration on Artificial Intelligence and Mathematics (2026)



Outline

01

Usage

AI running the model

Experiments, diagnostics, and differentiable simulations.

02

Development

AI writing the model

Implementation, review, and shared understanding.

03

Parameterization

AI inside the model

Calibration, learned closures, and online validation.

Usage

Designing experiments and interpreting model behavior

AI-assisted experiments

A useful agent workflow should produce evidence we can inspect.

Specify the scientific question and baseline.
Run controlled comparisons and record the configuration.
Inspect diagnostics and explain unexpected behavior.

Which decisions can an agent make—and which need us?

Related perspective: Held (2005), The Gap between Simulation and Understanding

Differentiating through a simulation

A single-column twin experiment recovers parameters from synthetic observations.

Oceananigans PR #5013 differentiates a simulation with Reactant + Enzyme and uses BFGS to estimate closure constants and surface fluxes.

Recovery in a twin experiment is a demonstration, not validation against the real ocean.

Oceananigans PR #5013 · merged 25 September 2026

Development

Writing code, reviewing changes, and retaining understanding

A short history of LLMs for coding

From code completion to coding agents

2021

Completion

GitHub Copilot preview: suggest code in the editor.

GitHub · Jun 2021

2022

Conversation

ChatGPT: explain, generate and debug code through chat.

OpenAI · Nov 2022

2023

In the editor

Copilot Chat reaches general availability in VS Code / VS.

GitHub · Dec 2023

2025

Agents

Claude Code + Codex: inspect repositories, edit files, run tests.

Anthropic · Feb; OpenAI · May

Selected milestones • assistants and agents coexist

The cost of implementation is falling

AI is pushing the cost of writing code toward zero.

For a well-specified feature, producing a candidate implementation can take very little developer time.

What happens when a community can write more code than it can evaluate?

Context: Anthropic (2026), “Bringing Code Review to Claude Code”

New contributors can enter complex codebases

Contributors can tackle changes before they have deep familiarity with the codebase.

AI helps them navigate existing code and assemble an implementation that follows local patterns.

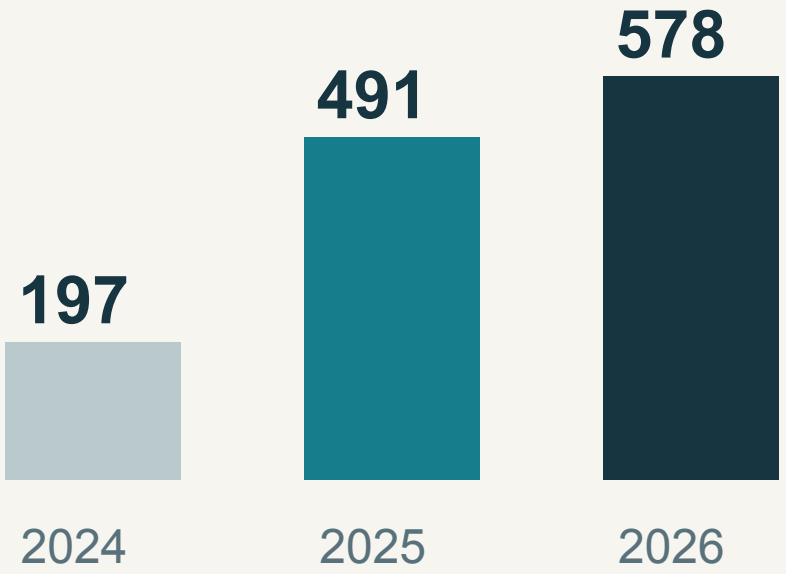
One first contribution with disclosed assistance: PR #5704.
First-time contributor does not mean novice programmer.

Tool capability: GitHub Docs, “Using GitHub Copilot to explore a codebase”

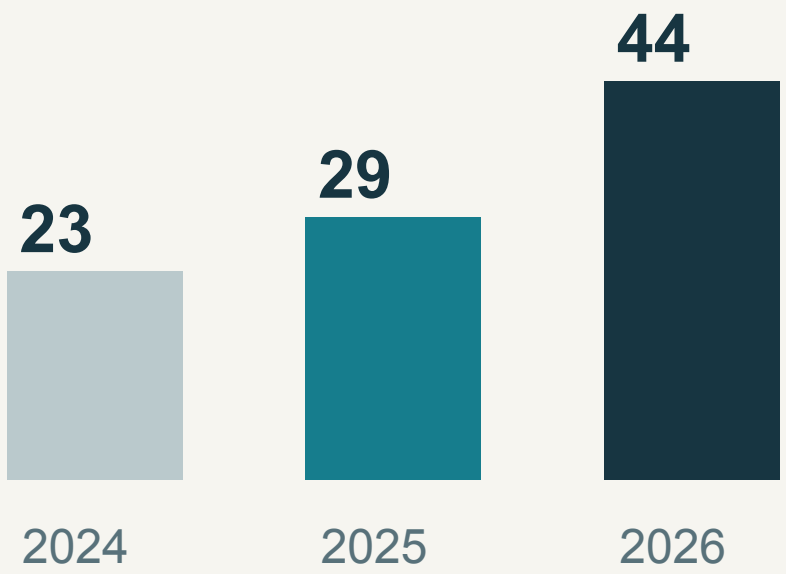
New contributors can enter complex codebases more easily

January–August comparisons; human-opened PRs

Oceananigans PRs opened



Distinct Oceananigans PR authors

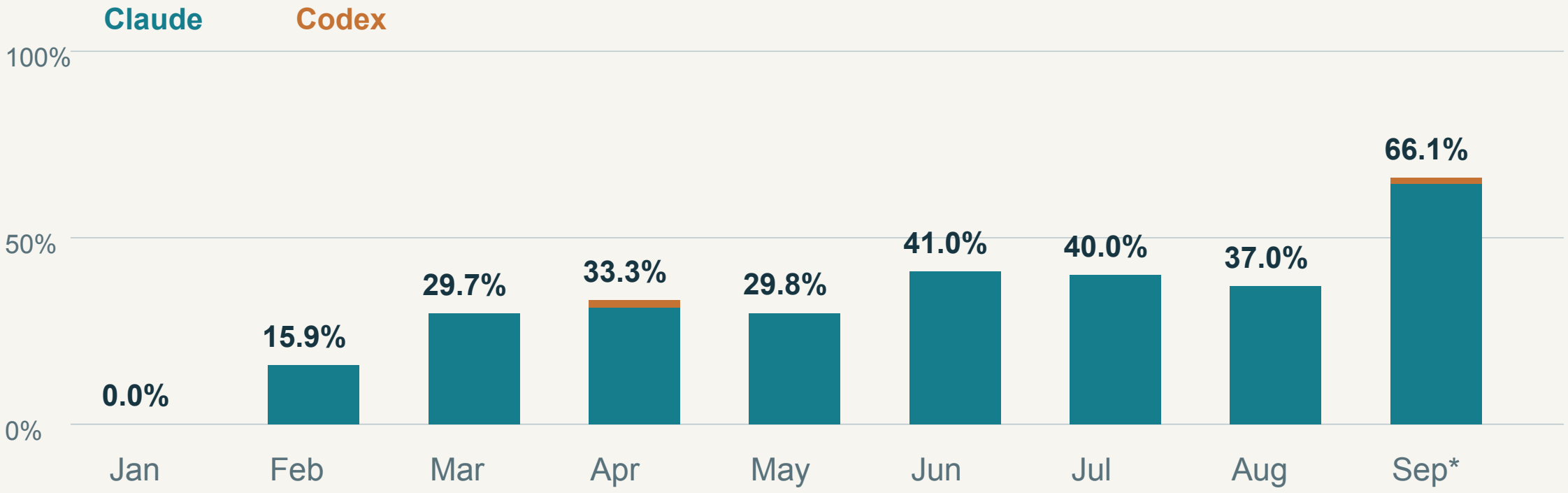


2025 → 2026: +18% PRs opened; +52% distinct authors

GitHub PR snapshot, 30 Sep 2026. Accounts ≠ people; growth does not establish an AI productivity effect.

Disclosed AI assistance is now common

2026 through snapshot: 204 / 597 commits (34.2%)



Co-author trailer OR linked PR disclosure; each commit counted once

Main history + PR descriptions, 30 Sep 2026. *September partial. Optional disclosure; not the fraction of AI-written code.

Review can become the bottleneck

More implementations arrive.
Maintainer time remains limited.

If we thoroughly vet every PR, our ability to review limits how quickly the model can develop.

Cheaper implementation can mean a longer review queue.

Related discussion: Google Cloud, “When AI writes the code, who reviews it?”

Also: Anthropic (2026), “Bringing Code Review to Claude Code”

AI-generated code still needs review

An Oceananigans contributor makes the division of labor explicit.

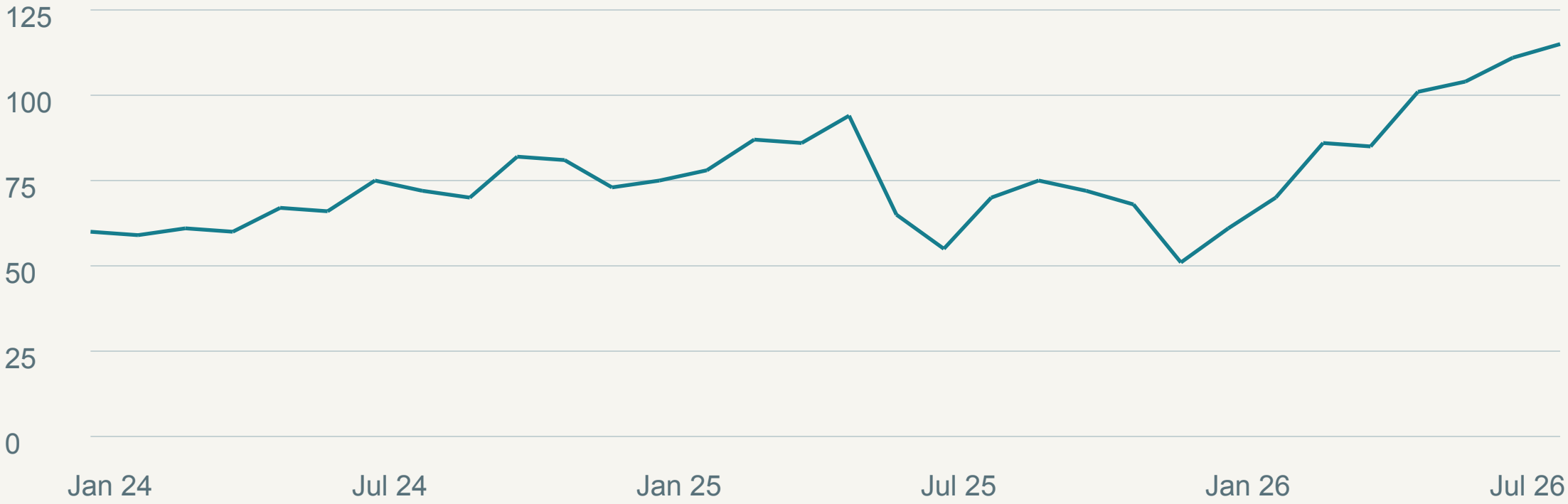
“Code and most of this PR description were generated with Claude Code.”

The author reviewed it, then requested another review.

Oceananigans PR #5742 · merged 26 August 2026

The open PR inventory has grown

51 at end-2025 → 115 at end-August 2026



A larger inventory is consistent with pressure on integration—not proof of a review bottleneck.

GitHub snapshot, 30 Sep 2026. Approximate inventory from creation/final closure dates; reopening history unavailable.

Understanding can become the bottleneck

Producing code is getting cheaper.
Understanding a model still takes work.

We need to connect the implementation to the equations, explain its behavior, and judge the effects of a change.

How quickly can our understanding keep pace?

Naur (1985), Programming as Theory Building

Unequal access can widen productivity gaps

Access to AI tools can widen existing productivity gaps.

Money buys tool access and capacity to use it.
Institutional support and training help turn access into gains.
Unequal access can amplify existing research inequalities.

Community question: should access to AI tools be shared research infrastructure?

ILO–World Bank (2024), Buffer or Bottleneck? GenAI and the digital divide
Brynjolfsson, Li & Raymond (2023), Generative AI at Work

What makes a change ready for review?

A candidate implementation still needs evidence.

What shows that the numerics are correct?
What tests or simulations demonstrate the intended behavior?
Can the contributor explain the change and its limitations?

How can contributors make that evidence easier to review?

Review guidance: Google Engineering Practices, “What to look for in a code review”

Parameterization

Calibrating closures and testing what works online

AI can assist numerical-method development

An Oceananigans example: rewriting WENO smoothness operators.

PR #5937 describes using Claude to recompute and verify coefficients for a difference formulation. The author reports fewer arithmetic operations.

Fewer operations are a promising mechanism.
A runtime speedup still needs a benchmark.

Oceananigans PR #5937 · merged 9 September 2026

Calibrating and learning closures

Calibration adjusts a chosen model.
Learning can also change its functional form.

How does the closure behave inside the simulation?
Does it respect the required physical constraints?
Where do calibration conditions stop being informative?

For NEMO: what online tests should a new closure pass?

Concrete calibration example: Oceananigans PR #5013

Three questions for NEMO

Usage

Which scientific decisions can we delegate to an agent?

Development

What evidence should accompany a contribution?

Parameterization

What tests establish that a closure works online?

Appendix

Working ideas and possible connections

Understanding scientific software
Knowledge, craft, and model hierarchies

Material to develop and weave into the main talk.

What the repository analysis measures

A fixed snapshot: 30 September 2026

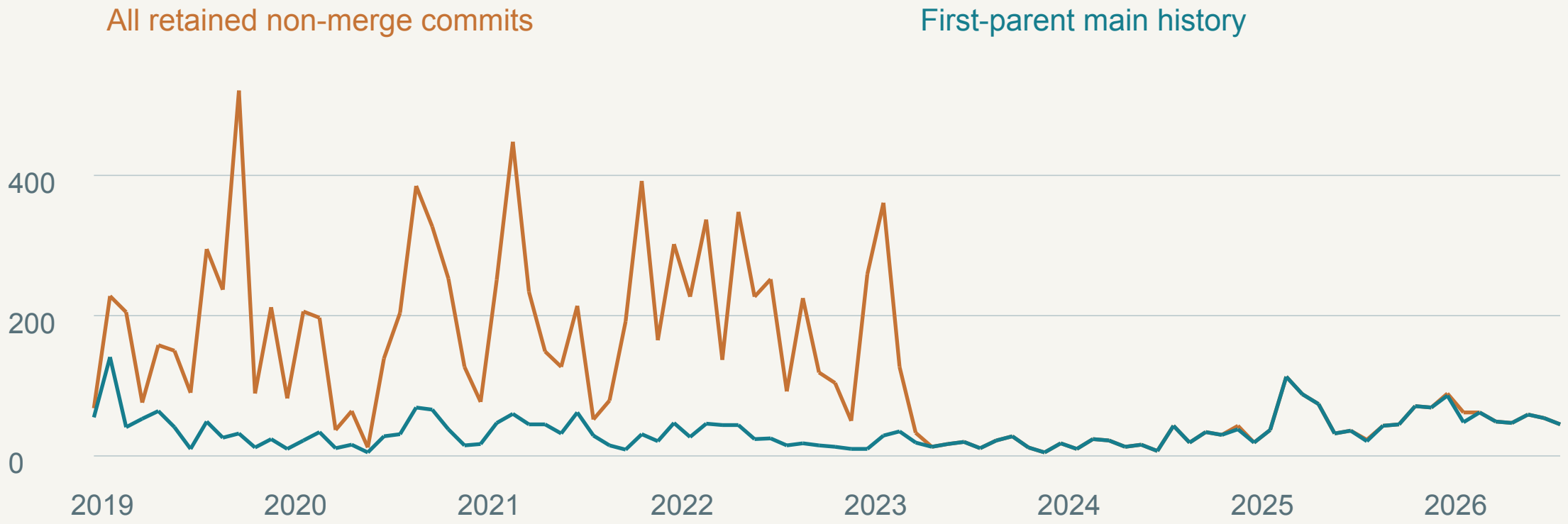
3,731 PRs and 13,845 commits reachable from main.
Growth comparisons use January–August; bots excluded.
Contributors are PR-opening accounts, not all developers.

Disclosure is optional. Activity is not quality or time saved.

Source: CliMA/Oceananigans.jl · methods and raw evidence saved with this talk

Why raw commit counts can mislead

Squashing changes how much development history a commit represents

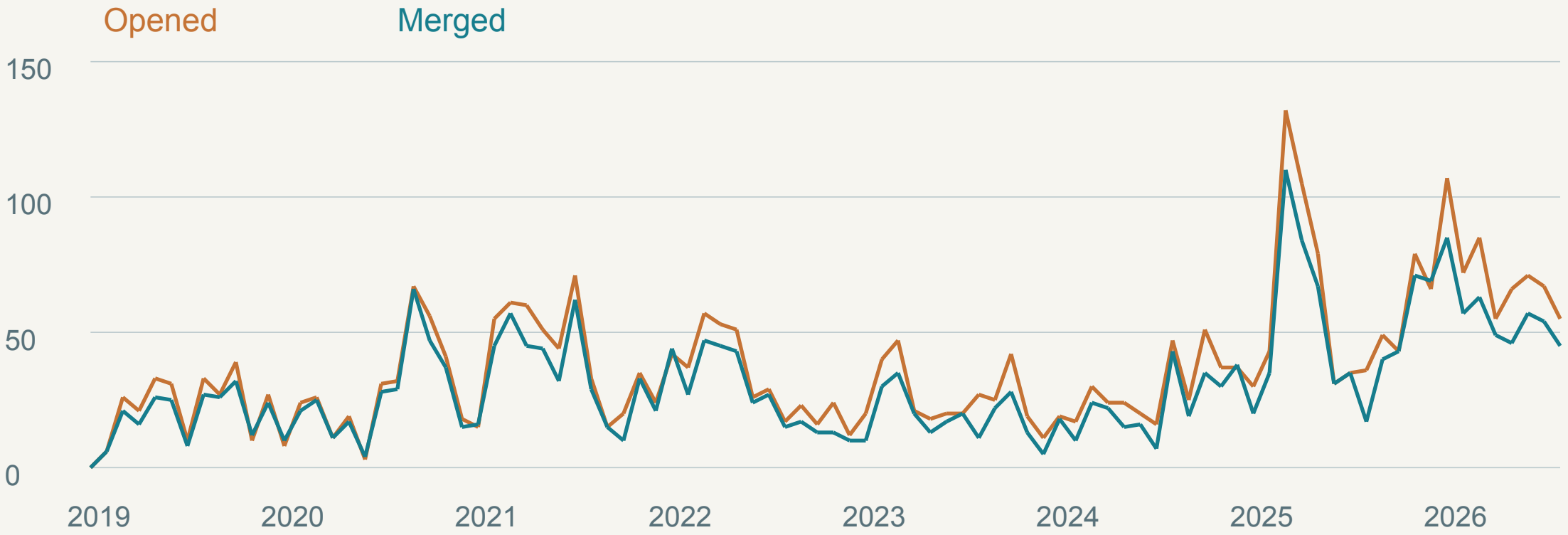


Use PRs as the primary activity measure; commit counts depend on merge workflow.

Git main history at b9725744cd31; 30 Sep 2026. Known bots excluded. UTC committer dates; Jan 2019–Aug 2026.

Development activity over time

Monthly human PR activity, January 2019–August 2026



Activity grew strongly in 2025; the timeline alone does not identify AI's effect.

GitHub PR snapshot, 30 Sep 2026. Bots excluded. Opened and merged counts use their respective event dates.

Programming builds understanding

The code is only part of what developers create.

Naur emphasizes the understanding behind a program:
how it represents a problem, why it has its particular form,
and how to adapt it when requirements change.

AI connection: generating code does not automatically transfer that understanding to its maintainers.

Naur (1985), Programming as Theory Building

Knowledge and craft are intertwined

Episteme: knowledge
Techne: craft or art

Parry cautions against a simple theory–practice split.
In Aristotle, craft can itself involve theoretical understanding.

Discussion: what understanding belongs to the craft of developing an Earth system model?

Parry, Episteme and Techne, Stanford Encyclopedia of Philosophy

Scientific tools help produce knowledge

People and instruments jointly shape scientific work.

Russo examines the role of technologies and instruments
in how scientific knowledge is produced.

**Discussion: if AI helps build our modeling tools,
how does that change what we learn by building them?**

Russo (2022), Techno-Scientific Practices: An Informational Approach

Understanding through simpler models

Model hierarchies help us identify mechanisms.

Held argues for closing the gap between simulation and understanding. Jeevanjee and colleagues show how simplification and mechanism tests can help.

AI connection: can agents help design and run the simpler experiments that explain a complex model?

Held (2005), The Gap between Simulation and Understanding in Climate Modeling
Jeevanjee et al. (2017), A perspective on climate model hierarchies

Who needs to understand what?

Understanding a component and understanding the whole model are different demands.

Which assumptions must the contributor explain?
Which interactions must a maintainer assess?
What knowledge must the community retain?

Working question: how can we preserve shared understanding as the codebase grows?

Conceptual starting point: Naur (1985), Programming as Theory Building

Can AI accelerate understanding too?

Implementation speed is only one possible benefit.

AI can help explore unfamiliar code and explain its structure.
Could it also help us test assumptions and investigate unexpected model behavior?

Possible comparison: time to produce a change and time to explain and validate it.

GitHub Docs, Using GitHub Copilot to explore a codebase